# AI Software Creation Prompt

## ChequePrint Pro — Indian Bank Cheque Printing Desktop Application

### Overview

You are tasked with building a **fully functional, production-ready desktop application** called **ChequePrint Pro** — a cheque printing software designed for Indian users, banks, businesses, and individuals. The software must support all major Indian bank cheque formats, allow precise alignment of printed text on physical cheque leaves, and work with any standard inkjet or laser printer (including Canon PIXMA MG2500 and similar consumer-grade printers).

Build this as an **Electron.js desktop application** (cross-platform: Windows, macOS, Linux) with a **React + Tailwind CSS** frontend and a **Node.js** backend for print management.

### Core Functional Requirements

#### 1. Bank Cheque Template Management

- Include pre-configured templates for all major Indian banks:
  - State Bank of India (SBI)
  - HDFC Bank
  - ICICI Bank
  - Punjab National Bank (PNB)
  - Bank of Baroda (BoB)
  - Axis Bank
  - Kotak Mahindra Bank
  - Canara Bank
  - Union Bank of India
  - IndusInd Bank
  - Yes Bank
  - IDBI Bank
  - Federal Bank
  - South Indian Bank
  - Bank of India

- Each template stores: cheque paper dimensions (width × height in mm), field positions (X, Y coordinates in mm), field sizes, and font sizes for each field.
- Allow users to create and save **custom templates** for unlisted banks by manually setting field positions via drag-and-drop on a live preview canvas.

## 2. Cheque Data Entry Form

Each cheque must capture:

- **Pay To (Payee Name)** — text field, max 80 characters
- **Amount in Numbers** — numeric field with Indian number formatting (e.g., ₹1,25,000)
- **Amount in Words** — auto-generated from numeric amount using Indian number-to-words conversion (handles lakhs, crores)
- **Date** — date picker with DD/MM/YYYY format (standard Indian cheque date)
- **Account Number** — optional reference field
- **IFSC Code** — optional
- **Memo / Narration** — optional note field (not printed on cheque, internal use)
- **Cheque Number** — optional auto-increment field for record keeping

## 3. Live Print Preview

- Real-time WYSIWYG preview canvas showing the cheque as it will print.
- Overlay actual cheque field positions on a neutral background representing the cheque leaf.
- Allow users to visually adjust X/Y offset of each field using arrow keys or numeric input.
- Preview must accurately reflect the selected paper size and field positions.
- Toggle grid overlay for alignment assistance.

## 4. Print Engine

- Send print jobs directly to the system printer using the OS print API.
- Support custom paper size matching the cheque leaf dimensions.
- Print in **portrait** or **landscape** orientation (configurable per bank template).
- Allow setting DPI (dots per inch) for print quality — default 300 DPI.
- Support **offset correction** (global left/top nudge in mm) for physical printer calibration.
- Print a single cheque or batch print multiple cheques in sequence.

### 5. Batch Cheque Printing

- Import cheque data from CSV or Excel (XLSX) file.
- Map CSV columns to cheque fields via a guided column-mapping UI.
- Review all imported cheques in a table before printing.
- Print all in batch or selectively mark individual cheques for printing.
- Export imported/generated cheque records as CSV for record-keeping.

### 6. Cheque Record Management

- Maintain a local SQLite database of all printed cheques.
- Fields stored: cheque number, bank, payee, amount, date, printed-on timestamp, status (pending/printed/cancelled).
- Search, filter, and export records.
- Mark cheques as cancelled with reason.

### 7. Printer Calibration Wizard

- Step-by-step wizard to calibrate printer offset:
  1. Print a test page on plain A4 paper.
  2. User places a cheque leaf over the printed test page and holds to light.
  3. User measures offset in mm (horizontal/vertical).
  4. Software saves the offset and applies to all future prints for that bank template.

### 8. Settings & Configuration

- Default bank selection
- Auto-increment cheque number
- Font face selection (Arial, Times New Roman, Courier New — all standard for cheques)
- Font size per field
- Date format (DD/MM/YYYY or DD-MM-YYYY)
- Amount formatting preference (₹ symbol or Rs.)
- Backup and restore database
- Export settings as JSON

## Technical Stack

| Layer | Technology |
|---|---|
| Shell | Electron.js v28+ |
| Frontend | React 18 + Vite |

| Layer | Technology |
|---|---|
| Styling | Tailwind CSS v3 |
| State Management | Zustand |
| Database | SQLite (via better-sqlite3) |
| Print Engine | electron-print / OS native print API |
| Number to Words | Custom Indian locale converter (handles lakhs and crores) |
| CSV/Excel Import | PapaParse (CSV) + xlsx.js (Excel) |
| PDF Export | jsPDF |
| Validation | Zod |
| Icons | Lucide React |

## UI/UX Design Requirements

### Application Layout

```
┌──────────────────────────────────────────────────────────┐
│  [Logo] ChequePrint Pro         [Settings] [Help]  │  ← Header/Navbar
├──────────────┬──────────────────────────────────────────┤
│              │                                      │
│  Sidebar Nav │   Main Content Area                  │
│  ──────────  │   (Preview | Form | Records | etc.)  │
│  ▣ Banks     │                                      │
│  ▣ New       │                                      │
│  ▣ Records   │                                      │
│  ▣ Batch     │                                      │
│  ▣ Calibrate │                                      │
│  ⚙ Settings  │                                      │
│              │                                      │
└──────────────┴──────────────────────────────────────┘
```

### Color Scheme

- Primary: Deep Teal `#01696f` (trust, banking)
- Background: Warm off-white `#f7f6f2`
- Accent: Amber `#d19900` for action buttons
- Error: Maroon `#a12c7b`
- Text: Dark charcoal `#28251d`

## Design Principles

- Clean, minimal UI — no decorative gradients or glowing elements
- All form labels must be visible and persistent (no placeholder-only labels)
- Large clickable targets (minimum 44×44px) for all buttons
- Keyboard-navigable throughout
- Responsive sidebar collapses on smaller screens

## Number to Words — Indian Locale

Implement a custom converter that handles Indian number system:

- Units: Zero to Nineteen
- Tens: Twenty to Ninety
- Hundred
- Thousand
- Lakh (1,00,000)
- Crore (1,00,00,000)

Example:

- `125000` → "One Lakh Twenty Five Thousand Only"
- `75050` → "Seventy Five Thousand And Fifty Only"
- `10000000` → "One Crore Only"

Append "Only" at the end of all amount-in-words strings (Indian banking standard).

## File Structure

```
chequeprintpro/
├── electron/
│   ├── main.js             # Electron main process
│   ├── preload.js          # IPC bridge
│   └── print.js            # Print engine
├── src/
│   ├── components/
│   │   ├── ChequeForm.jsx
│   │   ├── ChequePreview.jsx
│   │   ├── BankSelector.jsx
│   │   ├── BatchImport.jsx
│   │   ├── RecordsTable.jsx
│   │   ├── CalibrationWizard.jsx
│   │   └── Settings.jsx
│   ├── data/
│   │   └── bankTemplates.js  # Pre-configured bank templates
│   ├── lib/
│   │   ├── db.js            # SQLite operations
```

```
│   │   ├── numberToWords.js  # Indian locale converter
│   │   ├── printService.js   # Print job manager
│   │   └── csvImport.js      # CSV/Excel importer
│   ├── store/
│   │   └── appStore.js       # Zustand global state
│   ├── App.jsx
│   └── main.jsx
├── package.json
├── tailwind.config.js
├── vite.config.js
└── electron-builder.config.js
```

## Key Algorithms to Implement

### 1. Field Positioning Algorithm

Each cheque field has a position defined in millimetres from the top-left corner of the cheque leaf. When printing:

```
printX = (fieldX_mm + globalOffsetX_mm) * DPI / 25.4
printY = (fieldY_mm + globalOffsetY_mm) * DPI / 25.4
```

### 2. Cheque Validation Rules

- Payee name: alphanumeric + common punctuation, max 80 chars
- Amount in numbers: must be > 0, max ₹99,99,99,999 (Indian banking limit)
- Date: must not be more than 3 months in the past (post-dated/stale cheque check), warn user
- Amount in words must exactly match amount in numbers (cross-check before printing)

### 3. Batch Import Validation

- Validate each row before adding to queue
- Flag and skip rows with errors
- Show error count and allow user to fix before printing

## Security & Data Requirements

- All data stored locally — no cloud upload, no telemetry
- SQLite database stored in the OS user data directory
- No internet connection required after installation
- Backup/restore as encrypted ZIP (AES-256)

## Deliverables

1. Complete Electron application source code
2. All pre-configured bank templates (JS data file)
3. Indian number-to-words converter module
4. SQLite schema and migration scripts
5. Installer scripts for Windows (NSIS), macOS (DMG), Linux (AppImage)
6. README with setup instructions

## Constraints & Notes

- Must work offline (no API calls)
- Must support Windows 10/11, macOS 12+, Ubuntu 20.04+
- Installer size target: under 120 MB
- First-launch time: under 3 seconds
- Print latency: under 2 seconds from "Print" button click to print job dispatch
- Tested printer models: Canon PIXMA MG2500, HP DeskJet 2700, Epson L3250